



FOM Hochschule für Oekonomie & Management

Hochschulzentrum München

Seminararbeit

im Studiengang Informatik

2. Fachsemester

im Rahmen der Lehrveranstaltung

Wissenschaftliches Arbeiten

über das Thema

**Potenziale und Risiken des Einsatzes von KI-gestützten Code-Generatoren in
der agilen Softwareentwicklung**

von

Markus Grimmer

Betreuer : Prof. Dr. Holger Schmidt

Matrikelnummer : 854657

Abgabedatum : 27. Juli 2026

Inhaltsverzeichnis

Abkürzungsverzeichnis	III
Glossar	IV
1 Einleitung	1
2 Konzeptionelle Grundlagen	1
2.1 KI-gestützte Code-Generatoren: Funktionsweise und Einsatzformen	2
2.2 Agile Softwareentwicklung als Einsatzkontext	2
3 Potenziale des Einsatzes in agilen Entwicklungsprozessen	3
3.1 Produktivitäts- und Geschwindigkeitseffekte	3
3.2 Qualitätssicherungs- und Wissenseffekte	4
4 Risiken und Herausforderungen	4
4.1 Qualitäts- und Sicherheitsrisiken	4
4.2 Kompetenz-, prozess- und rechtsbezogene Risiken	5
5 Kritische Würdigung und Gestaltungsempfehlungen	6
6 Fazit und Ausblick	7
Anhang	8
Literaturverzeichnis	9

Abkürzungsverzeichnis

DORA DevOps Research and Assessment

HTTP Hypertext Transfer Protocol

KI Künstliche Intelligenz

LLM Large Language Model

Glossar

Agentisches Werkzeug KI-gestütztes Werkzeug, das in der Entwicklungsumgebung selbstständig mehrere Arbeitsschritte ausführt, also Dateien durchsucht, ändert und Programme iterativ testet.

Agile Softwareentwicklung Vorgehensweise, die Software in kurzen Abständen ausliefert, Anforderungen fortlaufend anpasst und das eigene Vorgehen regelmäßig reflektiert.

Code Review Systematische Prüfung von Quellcode durch andere Personen als die verfassende, um Fehler und Qualitätsmängel vor der Übernahme aufzudecken.

Code-Generator Werkzeug, das auf Basis eines großen Sprachmodells aus dem umgebenden Kontext Quellcode vorschlägt oder erzeugt.

Debugging Systematisches Auffinden, Eingrenzen und Beheben von Fehlern in einem Programm.

Definition of Done Formale Beschreibung des Zustands, den ein Increment erreichen muss, um als fertiggestellt zu gelten; dient als gemeinsamer Qualitätsmaßstab des Teams.

DevOps Zusammenführung von Softwareentwicklung (Development) und IT-Betrieb (Operations) mit dem Ziel, Änderungen häufig und zuverlässig auszuliefern.

Extreme Programming Agiles Vorgehensmodell, das die Entwicklung über zwölf Vorgehensweisen wie Pair Programming, testgetriebene Entwicklung und gemeinsame Verantwortlichkeit für den Quellcode ordnet.

Increment In sich nutzbares Zwischenergebnis, das ein Team am Ende eines Sprints zur bisherigen Produktversion hinzufügt.

Konfidenzintervall Statistischer Wertebereich, der den geschätzten Kennwert mit einer festgelegten Wahrscheinlichkeit überdeckt und damit die Genauigkeit der Schätzung ausdrückt.

Open Source Software, deren Quellcode öffentlich zugänglich ist und unter einer Lizenz steht, die Nutzung, Weitergabe und Veränderung erlaubt.

Overreliance Übermäßiges Vertrauen in die Ausgaben eines KI-Werkzeugs, das dazu führt, dass fehlerhafte Vorschläge ungeprüft übernommen werden.

Pair Programming Praktik, bei der zwei Entwickelnde gemeinsam an einem Arbeitsplatz denselben Quellcode schreiben und dabei fortlaufend die Rollen tauschen.

Repository Versioniert verwaltetes Verzeichnis, in dem der Quellcode eines Projekts samt seiner Änderungshistorie abgelegt wird.

Retrospektive Regelmäßiger Termin am Ende eines Sprints, in dem das Team das eigene Vorgehen überprüft und Verbesserungen beschließt.

Scrum Rahmenwerk der agilen Softwareentwicklung, das die Arbeit in Sprints strukturiert und Rollen, Termine und Artefakte festlegt.

Sprint Fester Zeitraum von höchstens einem Monat, in dem ein Scrum-Team ein nutzbares Increment fertigstellt.

Sprint Review Termin am Ende eines Sprints, in dem das Increment gemeinsam mit den Beteiligten geprüft und das weitere Vorgehen abgestimmt wird.

Story Point Relative Schätzgröße für den Umfang einer Anforderung, die statt absoluter Zeitangaben den Aufwand im Vergleich zu Referenzanforderungen ausdrückt.

Unit-Test Automatisierter Test, der eine einzelne Programmeinheit isoliert gegen ihre erwartete Funktionsweise prüft.

Velocity Kennzahl der je Iteration fertiggestellten Story Points; sie dient der realistischen Vorhersage künftiger Iterationen, nicht der Leistungsbewertung.

1 Einleitung

Große Sprachmodelle sind in der Softwareentwicklung angekommen: 81 % der Befragten des DORA-Reports 2024 berichten eine Prioritätenverschiebung zugunsten von KI.¹ Eine Mehrfallstudie in drei deutschen Organisationen zeigt den regulären Einsatz in agilen Teams bei häufig fehlenden Regelungen.² Die Modelle treffen dort auf ein Vorgehensmodell aus kurzen Lieferzyklen, technischer Exzellenz und regelmäßiger Reflexion.³

Die Befundlage ist widersprüchlich. Ein Laborexperiment weist eine um 55,8 % kürzere Bearbeitungszeit aus, ein randomisierter Versuch mit erfahrenen Entwicklern dagegen eine um 19 % längere.⁴ Der DORA-Report verbindet zudem eine um 25 % höhere KI-Adoption mit besserer Dokumentationsqualität, aber geringerer Auslieferungsstabilität.⁵ Ob Code-Generatoren agile Prozesse stärken oder aushöhlen, ist damit offen.

Die Arbeit fragt, welche Potenziale und Risiken der Einsatz KI-gestützter Code-Generatoren in agilen Softwareentwicklungsprozessen mit sich bringt und welche Gestaltungsempfehlungen daraus folgen. Ziel ist deren Systematisierung entlang agiler Praktiken.

Methodisch stützt sich die Arbeit auf eine literaturbasierte Studienauswertung; eigene Erhebungen, Werkzeugvergleiche, juristische Detailfragen und KI-Ethik bleiben ausgeklammert. Kapitel 2 klärt die Begriffe, Kapitel 3 und 4 stellen Potenziale und Risiken gegenüber, Kapitel 5 wägt ab, Kapitel 6 beantwortet die Forschungsfrage.

2 Konzeptionelle Grundlagen

Die Analyse setzt zwei Klärungen voraus: was unter einem KI-gestützten Code-Generator zu verstehen ist und welche agilen Praktiken den Bezugsrahmen bilden, an dem die folgenden Kapitel die Wirkungen prüfen.

¹ Vgl. *DORA, State of DevOps*, 2024, S. 18.

² Vgl. *Neumann, M. et al., GenAI Adoption*, 2026, S. 297 f.

³ Vgl. *Beck, K. et al., Agile Prinzipien*, 2001, o. S.

⁴ Vgl. *Peng, S. et al., Developer Productivity*, 2023, S. 2; *Becker, J. et al., AI Impact*, 2025, S. 2.

⁵ Vgl. *DORA, State of DevOps*, 2024, S. 37, S. 39.

2.1 KI-gestützte Code-Generatoren: Funktionsweise und Einsatzformen

Als KI-gestützte Code-Generatoren gelten hier Werkzeuge auf Basis großer Sprachmodelle (Large Language Models, LLM), die auf umfangreichen Mengen quelloffenen Codes trainiert sind und aus dem Kontext – Kommentaren, Funktionsnamen, umgebendem Code – Vervollständigungen vorschlagen.⁶ Verbreitet sind mehrere Modellfamilien, darunter GPT-4o, Claude 3.5, Gemini 1.5, Codestral und Llama 3.⁷ Zu unterscheiden sind drei Einsatzformen: die dialogische Nutzung über eine Weboberfläche, die Integration in die Entwicklungsumgebung und agentische Werkzeuge, die dort selbstständig Dateien durchsuchen, ändern und Programme iterativ debuggen.⁸ Die Unterscheidung ist keine Formalie: Studien fixieren jeweils eine Form. Eine Qualitätsuntersuchung bildet dabei bewusst die kostenfreie Weboberfläche nach, wie sie besonders Berufseinsteiger nutzen.⁹ Ihre Befunde gelten folglich nicht ohne Weiteres für den in Entwicklungsumgebungen integrierten Einsatz.

2.2 Agile Softwareentwicklung als Einsatzkontext

Agile Entwicklung liefert Software in kurzen Abständen und reflektiert regelmäßig das eigene Vorgehen.¹⁰ In Scrum verdichtet sich das zu Sprints von höchstens einem Monat, in denen die Qualität nicht abnehmen darf.¹¹ Als Qualitätsmaßstab dient die Definition of Done, eine formale Beschreibung des Zustands, den ein Increment erreichen muss; konkretisiert wird sie über Kriterien wie fehlerfreie Unit-Tests, ersatzweise über das Vier-Augen-Prinzip.¹² Zwei getrennte Schleifen prüfen das Erreichte: das Sprint Review das Produkt, die Retrospektive den Prozess.¹³ Verantwortlich ist das Team, das für das Increment entsteht und selbst entscheidet, wie Anforderungen umgesetzt werden.¹⁴ Hinzu treten Praktiken mit hohem Anteil sozialer Interaktion wie Pair Programming¹⁵ sowie Code

⁶ Vgl. *Pearce, H. et al.*, *Asleep at the Keyboard*, 2022, S. 754.

⁷ Vgl. *Kharm, M. F. et al.*, *LLM-Generated Code*, 2026, S. 7300.

⁸ Vgl. *Becker, J. et al.*, *AI Impact*, 2025, S. 6.

⁹ Vgl. *Swaraj, A., Kumar, S.*, *Trust But Verify*, 2026, S. 67.

¹⁰ Vgl. *Beck, K. et al.*, *Agile Prinzipien*, 2001, o. S.

¹¹ Vgl. *Schwaber, K., Sutherland, J.*, *Scrum Guide*, 2020, S. 8.

¹² Vgl. *Schwaber, K., Sutherland, J.*, *Scrum Guide*, 2020, S. 13; *Preußig, J.*, *Agiles Projektmanagement*, 2020, S. 93.

¹³ Vgl. *Schwaber, K., Sutherland, J.*, *Scrum Guide*, 2020, S. 10 f.

¹⁴ Vgl. ebd., S. 5, S. 9.

¹⁵ Vgl. *Neumann, M. et al.*, *GenAI Adoption*, 2026, S. 291.

Reviews, deren Geschwindigkeit als Prozessgröße erhoben wird.¹⁶ Diese sechs Elemente bilden das Raster für die Kapitel 3 und 4.

3 Potenziale des Einsatzes in agilen Entwicklungsprozessen

Die berichteten Vorteile liegen auf zwei Ebenen: in der Geschwindigkeit der Erstellung und in der Qualitätssicherung samt Wissensweitergabe. Beide werden am Raster aus Kapitel 2.2 geprüft.

3.1 Produktivitäts- und Geschwindigkeitseffekte

Ein kontrolliertes Experiment mit 95 rekrutierten Programmierern zeigt den Effekt unter Laborbedingungen: Bei der Implementierung eines HTTP-Servers in JavaScript arbeitete die Gruppe mit Assistenz 55,8 % schneller, bei einem Konfidenzintervall von 21 % bis 89 % und dem stärksten Effekt bei weniger erfahrenen Teilnehmern.¹⁷ Ein Feldexperiment in einem Technologiekonzern bestätigt die Richtung unter Praxisbedingungen: Nach Einführung eines internen Sprachmodells stieg der Code-Ausstoß um 55 %, bei Beschäftigten mit höchstens einem Jahr Erfahrung um 67 %, während der Effekt bei erfahreneren Beschäftigten nicht signifikant war.¹⁸ Gemessen wird dort die Menge produzierter Codezeilen, nicht deren Qualität; nur 11 bis 18 Prozentpunkte des Zuwachses entfallen auf unmittelbar übernommene Modellausgaben, die übrigen 36 bis 43 führen die Autoren auf Zeitersparnis zurück.¹⁹ Dem steht ein randomisierter Versuch entgegen, in dem 16 erfahrene Open-Source-Entwickler 246 Aufgaben in ihren eigenen Repositorien bearbeiteten: Sie benötigten mit Assistenz 19 % mehr Zeit, hatten vorab aber eine Beschleunigung um 24 % erwartet.²⁰ Die Befunde widersprechen einander nur scheinbar, denn sie unterscheiden sich systematisch in Erfahrung und Vertrautheit mit der Codebasis. Daraus folgt für diese Arbeit: Der Zeitgewinn konzentriert sich dort, wo Kontextwissen fehlt. Ein pauschaler Zuwachs der Velocity, also der je Iteration fertiggestellten Story Points,²¹ ist nicht zu erwarten.

¹⁶ Vgl. *DORA*, State of DevOps, 2024, S. 37.

¹⁷ Vgl. *Peng, S. et al.*, Developer Productivity, 2023, S. 2 f.

¹⁸ Vgl. *Gambacorta, L. et al.*, Labour Productivity, 2024, S. 5.

¹⁹ Vgl. ebd., S. 5.

²⁰ Vgl. *Becker, J. et al.*, AI Impact, 2025, S. 2.

²¹ Vgl. *Preußig, J.*, Agiles Projektmanagement, 2020, S. 132.

3.2 Qualitätssicherungs- und Wissenseffekte

Auf Prozessebene verbindet der DORA-Report eine um 25 % höhere KI-Adoption mit einer um 7,5 % besseren Dokumentationsqualität, einer um 3,4 % höheren Codequalität und um 3,1 % schnelleren Code Reviews.²² Diese Größen lassen sich dem Raster aus Kapitel 2.2 zuordnen: Die Review-Geschwindigkeit betrifft die Praktik Code Review unmittelbar, die Codequalität den Maßstab der Definition of Done. Eine Befragung in zwei großen agilen Organisationen stützt das aus Nutzersicht: Zwei Drittel berichten geringeren Aufwand für wiederkehrende Aufgaben, drei Fünftel weniger Zeit für die Informationssuche; häufiger im Arbeitsfluss bleibt jedoch nur knapp ein Drittel – deutlich weniger als in einer früheren Herstellerbefragung.²³ In agilen Teams reichen die Anwendungen über das Programmieren hinaus: Von 21 dokumentierten Anwendungsfällen entfallen sieben auf Dokumentation und sechs auf kreative Aufgaben, darunter die Vorbereitung von Retrospektiven; zugleich ersetzt das Werkzeug die Recherche in Foren.²⁴ Das senkt Einstiegshürden, bleibt aber an die einzelne Person gebunden: Dieselbe Untersuchung beschreibt die Werkzeuge als persönliche Produktivitätsassistenten, die einzelne Tätigkeiten unterstützt, ohne die Zusammenarbeit im Team zu verändern.²⁵

4 Risiken und Herausforderungen

Den Potenzialen stehen Befunde gegenüber, die dieselben Praktiken betreffen. Sie ordnen sich in zwei Gruppen: Qualität und Sicherheit des erzeugten Codes sowie Wirkungen auf Kompetenz, Prozess und Recht.

4.1 Qualitäts- und Sicherheitsrisiken

Der erzeugte Code ist nicht durchgängig sicher. Eine Untersuchung von 1.689 durch einen Assistenten erzeugten Programmen wies rund 40 % als verwundbar aus.²⁶ Eine Messstudie über 200 Aufgaben, vier Sprachen und fünf Modellfamilien zeigt zudem Sprachabhängigkeit: In C und C++ treten Speicherfehler, fest eingebettete Geheimnisse und kryptografische Fehlanwendungen häufiger auf, moderne Sicherheitsmerkmale bleiben

²² Vgl. *DORA, State of DevOps*, 2024, S. 37.

²³ Vgl. *Wivestad, V. T., Ulfesnes, R.*, *Journey Through SPACE*, 2025, S. 46.

²⁴ Vgl. *Neumann, M.* et al., *GenAI Adoption*, 2026, S. 298–300.

²⁵ Vgl. ebd., S. 299.

²⁶ Vgl. *Pearce, H.* et al., *Asleep at the Keyboard*, 2022, S. 754.

oft ungenutzt.²⁷ In einer Nutzerstudie mit 47 Teilnehmern schrieb die Gruppe mit Assistenz signifikant unsichereren Code und hielt ihn zugleich häufiger für sicher.²⁸ Solches Übernehmen fehlerhafter Modellausgaben wird als Overreliance bezeichnet.²⁹ Hinzu tritt der Korrekturaufwand für Vorschläge, die fast richtig sind: 66 % der Befragten einer Entwicklerumfrage nennen ihn als Hauptärgernis, 45 % das Debuggen KI-erzeugten Codes.³⁰ Auf Prozessebene sinken Auslieferungsstabilität (−7,2 %) und Durchsatz (−1,5 %) je 25 % höherer KI-Adoption; als Erklärung vermutet der Report größere Änderungspakete und erinnert an das Prinzip kleiner Losgrößen.³¹ Daraus ergibt sich eine Rückkopplung: Code Review und Definition of Done sollen solche Fehler abfangen, setzen aber die prüfende Aufmerksamkeit voraus, die Overreliance gerade schwächt.

4.2 Kompetenz-, prozess- und rechtsbezogene Risiken

Die zweite Gruppe betrifft Menschen und Prozesse. Ein randomisiertes Experiment zum Erlernen einer unbekannten Programmbibliothek – durchgeführt von einem Modellanbieter und bislang als Vorabfassung veröffentlicht – findet, dass KI-Unterstützung Verständnis, Code-Lesen und Debugging beeinträchtigt, ohne im Mittel Zeit zu sparen; wer die Aufgabe vollständig delegierte, gewann Produktivität auf Kosten des Lernens.³² Dazu passt, dass rund 20 % der befragten Entwickler geringeres Vertrauen in die eigenen Fähigkeiten berichten und nur 17 % eine bessere Zusammenarbeit im Team beobachten.³³ Daraus folgt ein Risiko für die gemeinsame Verantwortlichkeit, eine der zwölf Vorgehensweisen des Extreme Programming:³⁴ Sie setzt voraus, dass Teammitglieder fremden Code lesen und ändern können – genau die Fähigkeit, die nach diesen Befunden leidet. Hinzu kommt eine Wahrnehmungslücke: Dieselben Entwickler, die 19 % länger brauchten, schätzten sich im Nachhinein um 20 % schneller ein; Aufwandsschätzungen und Velocity-Werte dürften dadurch verzerrt werden.³⁵ Rechtlich und organisatorisch bleibt die Vertraulichkeit offen. Fehlen verbindliche Richtlinien, entscheiden Beschäftigte nach eigenem Ermessen; Code mit Schlüsseln oder personenbezogenen Daten darf Werkzeugen ohne gesicherte Datenhoheit nicht übergeben werden.³⁶

²⁷ Vgl. *Khurma, M. F. et al.*, LLM-Generated Code, 2026, S. 7300.

²⁸ Vgl. *Perry, N. et al.*, Insecure Code, 2023, S. 2785.

²⁹ Vgl. *Shen, J. H., Tamkin, A.*, Skill Formation, 2026, S. 4.

³⁰ Vgl. *Swaraj, A., Kumar, S.*, Trust But Verify, 2026, S. 71.

³¹ Vgl. *DORA*, State of DevOps, 2024, S. 39 f.

³² Vgl. *Shen, J. H., Tamkin, A.*, Skill Formation, 2026, S. 1.

³³ Vgl. *Swaraj, A., Kumar, S.*, Trust But Verify, 2026, S. 71.

³⁴ Vgl. *Preußig, J.*, Agiles Projektmanagement, 2020, S. 152 f.

³⁵ Vgl. *Becker, J. et al.*, AI Impact, 2025, S. 2.

³⁶ Vgl. *Neumann, M. et al.*, GenAI Adoption, 2026, S. 298.

5 Kritische Würdigung und Gestaltungsempfehlungen

Das Muster aus Kapitel 3.1 setzt sich fort: Die Befunde unterscheiden sich darin, worin die Werkzeuge eingebettet sind. Der DORA-Report zieht aus seinen eigenen gegenläufigen Zahlen den Schluss, dass ein verbesserter Entwicklungsprozess die Auslieferung nicht von selbst verbessert, solange Grundlagen wie kleine Änderungspakete und belastbare Testmechanismen fehlen.³⁷ Die Mehrfallstudie stützt das von der Teamseite: Die Werkzeuge wirken als persönliche Assistenz, ohne die Zusammenarbeit zu verändern.³⁸ Daraus ergibt sich die These dieser Arbeit: Über Nutzen und Schaden entscheidet nicht das Werkzeug, sondern die Praktik, die es umgibt. Review, Definition of Done und Retrospektive sind zugleich Verstärker der Potenziale und Schutz vor den Risiken.

Fünf Empfehlungen folgen daraus. Erstens ist die Definition of Done um Kriterien für generierten Code zu erweitern; angesichts der Verwundbarkeitsquoten aus Kapitel 4.1 gehört eine automatisierte Sicherheitsprüfung zur Mindestanforderung, und im Review ist generierter Code als solcher auszuweisen. Zweitens sind Nutzungsrichtlinien verbindlich zu fassen, da Beschäftigte sonst nach eigenem Ermessen entscheiden.³⁹ Drittens ist der Einsatz nach Erfahrungsstand zu differenzieren: Weil die Gewinne bei Berufseinsteigern auftreten und bei erfahrenen Entwicklern ausbleiben oder ins Gegenteil umschlagen,⁴⁰ liegt deren Beitrag eher in Architektur und Review als in generierungsnaher Arbeit. Viertens erhalten drei von sechs beobachteten Interaktionsmustern das Lernen auch unter KI-Nutzung;⁴¹ maßgeblich ist demnach die kognitive Beteiligung, nicht der Verzicht. Fünftens lässt sich der Wahrnehmungslücke aus Kapitel 4.2 mit dem vorhandenen Instrumentarium begegnen: Die Retrospektive ist die Praktik, die die Selbsteinschätzung korrigiert – allerdings nur, wenn ihr gemessene Durchlaufzeiten zugrunde liegen und nicht Eindrücke. Das entspricht dem Zweck der Größe: Team Velocity dient realistischen Vorhersagen, nicht der Maximierung der Arbeitsgeschwindigkeit.⁴²

Die Grenzen der Arbeit sind zu benennen. Die Literaturbasis ist jung und teils vorläufig: Zwei tragende Studien liegen als Vorabfassung vor, eine stammt vom Anbieter selbst, eine weitere umfasst sechzehn Personen. Laborbefunde und Selbstauskünfte lassen sich nicht ohne Weiteres auf agile Teams übertragen, und der Ursachenzusammenhang hinter den Prozesskennzahlen ist im Report selbst nur eine Vermutung. Die Empfehlungen sind daher begründete Ableitungen, keine gesicherten Wirkungsaussagen.

³⁷ Vgl. *DORA, State of DevOps*, 2024, S. 40.

³⁸ Vgl. *Neumann, M. et al., GenAI Adoption*, 2026, S. 299.

³⁹ Vgl. ebd., S. 298.

⁴⁰ Vgl. *Gambacorta, L. et al., Labour Productivity*, 2024, S. 5; *Becker, J. et al., AI Impact*, 2025, S. 2.

⁴¹ Vgl. *Shen, J. H., Tamkin, A., Skill Formation*, 2026, S. 1.

⁴² Vgl. *Preußig, J., Agiles Projektmanagement*, 2020, S. 133.

6 Fazit und Ausblick

Die Forschungsfrage lässt sich verdichtet beantworten. Die Potenziale liegen in beschleunigter Erstellung, besserer Dokumentation und niedrigeren Einstiegshürden, die Risiken in unsicherem Code, Overreliance, nachlassender Kompetenz und verzerrter Selbsteinschätzung. Beide Seiten treffen dieselben Praktiken. Ob Nutzen oder Schaden überwiegt, entscheidet sich daher nicht an der Werkzeugwahl, sondern daran, ob Review, Definition of Done und Retrospektive an generierten Code angepasst werden; die Empfehlungen aus Kapitel 5 setzen dort an.

Offen bleiben drei Fragen. Erstens die Langzeitwirkung auf die Kompetenzentwicklung, da die vorliegenden Untersuchungen kurze Zeiträume abbilden. Zweitens die Wirkung agentischer Werkzeuge: Sie ändern nach Kapitel 2.1 selbstständig Dateien, sodass die menschliche Prüfung erst nach mehreren Arbeitsschritten ansetzt. Drittens, ob sich die Rolle der Entwicklung dauerhaft vom Ausführen zum Überwachen verschiebt – eine Verschiebung, die für frühere Automatisierungswellen bereits beschrieben ist.⁴³

⁴³ Vgl. Shen, J. H., Tamkin, A., Skill Formation, 2026, S. 1.

Anhang

Anhang 1: KI-Hilfsmittelverzeichnis

Claude, Version Opus 4.8 (Anthropic) – Themenfindung und Strukturentwurf; eingesetzt am 28.06.2026.

Literaturverzeichnis

- Becker, Joel, Rush, Nate, Barnes, Beth, Rein, David* (AI Impact, 2025): Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity, arXiv:2507.09089, o. O., 2025, <<https://arxiv.org/abs/2507.09089>> [Zugriff: 2026-07-26]
- Gambacorta, Leonardo, Qiu, Han, Shan, Shuo, Rees, Daniel M.* (Labour Productivity, 2024): Generative AI and Labour Productivity: A Field Experiment on Coding, BIS Working Papers Nr. 1208, Basel: Bank for International Settlements, 2024
- Goldman, Alfredo, Marczak, Sabrina, Tonin, Graziela Simone, Bassi, Dairton, Silva da Silva, Tiago, Schön, Eva-Maria, Neumann, Michael, Pinna, Andrea* (Hrsg.) (XP 2026, 2026): Agile Processes in Software Engineering and Extreme Programming, LNBIP 578, Cham: Springer, 2026
- Kharma, Mohammed F., Choi, Soohyeon, Alkhanafseh, Mohammad, Mohaisen, David* (LLM-Generated Code, 2026): Security and Quality in LLM-Generated Code: A Multi-Language, Multi-Model Analysis, in: IEEE Transactions on Dependable and Secure Computing, 23 (2026), Nr. 3, S. 7300–7314
- Marchesi, Lodovica, Goldman, Alfredo, Lunesu, Maria Ilaria, Przybyłek, Adam, Aguiar, Ademar, Morgan, Lorraine, Wang, Xiaofeng, Pinna, Andrea* (Hrsg.) (XP 2024 Workshops, 2025): Agile Processes in Software Engineering and Extreme Programming – Workshops, LNBIP 524, Cham: Springer, 2025
- Neumann, Michael, Bischof, Lasse, Hinz, Nic Elias, Altun, Abdullah, Stockmann, Luca, Schrader, Dennis, Ahaus, Ana Carolina, Demirci, Erim Can, Gabel, Benjamin, Rauschenberger, Maria, Diebold, Philipp, Fritzemeier, Henning, Przybyłek, Adam* (GenAI Adoption, 2026): Between Policy and Practice: GenAI Adoption in Agile Software Development Teams, in: *Goldman, Alfredo, Marczak, Sabrina, Tonin, Graziela Simone, Bassi, Dairton, Silva da Silva, Tiago, Schön, Eva-Maria, Neumann, Michael, Pinna, Andrea* (Hrsg.), Agile Processes in Software Engineering and Extreme Programming, LNBIP 578, o. O.: Springer, 2026, S. 290–308
- Pearce, Hammond, Ahmad, Baleegh, Tan, Benjamin, Dolan-Gavitt, Brendan, Karri, Ramesh* (Asleep at the Keyboard, 2022): Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions, in: 2022 IEEE Symposium on Security and Privacy (SP), Piscataway: IEEE, 2022, S. 754–768
- Peng, Sida, Kalliamvakou, Eirini, Cihon, Peter, Demirer, Mert* (Developer Productivity, 2023): The Impact of AI on Developer Productivity: Evidence from GitHub Copilot, arXiv:2302.06590, o. O., 2023, <<https://arxiv.org/abs/2302.06590>> [Zugriff: 2026-07-26]

- Perry, Neil, Srivastava, Megha, Kumar, Deepak, Boneh, Dan* (Insecure Code, 2023): Do Users Write More Insecure Code with AI Assistants?, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23), New York: ACM, 2023, S. 2785–2799
- Preußig, Jörg* (Agiles Projektmanagement, 2020): Agiles Projektmanagement. Agilität und Scrum im klassischen Projektumfeld, 2. Aufl., Freiburg: Haufe-Lexware, 2020
- Shen, Judy Hanwen, Tamkin, Alex* (Skill Formation, 2026): How AI Impacts Skill Formation, arXiv:2601.20245, o. O., 2026, <<https://arxiv.org/abs/2601.20245>> [Zugriff: 2026-07-26]
- Swaraj, Aman, Kumar, Sandeep* (Trust But Verify, 2026): Trust But Verify: Analyzing the Tradeoffs of AI-Generated Code, in: IEEE Software, 43 (2026), Nr. 4, S. 66–75
- Wivestad, Viggo Tellefsen, Ulfesnes, Rasmus* (Journey Through SPACE, 2025): A Journey Through SPACE: Unpacking the Perceived Productivity of GitHub Copilot, in: *Marchesi, Lodovica, Goldman, Alfredo, Lunesu, Maria Ilaria, Przybyłek, Adam, Aguiar, Ademar, Morgan, Lorraine, Wang, Xiaofeng, Pinna, Andrea* (Hrsg.), Agile Processes in Software Engineering and Extreme Programming – Workshops, LNBIP 524, o. O.: Springer, 2025, S. 42–50

Internetquellen

Beck, Kent, Beedle, Mike, van Bennekum, Arie, Cockburn, Alistair, Cunningham, Ward, Fowler, Martin, Grenning, James, Highsmith, Jim, Hunt, Andrew, Jeffries, Ron, Kern, Jon, Marick, Brian, Martin, Robert C., Mellor, Steve, Schwaber, Ken, Sutherland, Jeff, Thomas, Dave (Agile Prinzipien, 2001): Prinzipien hinter dem Agilen Manifest, <<https://agilemanifesto.org/iso/de/principles.html>> (2001) [Zugriff: 2026-07-26]

DORA / Google Cloud (State of DevOps, 2024): Accelerate State of DevOps Report 2024, <<https://dora.dev/research/2024/dora-report/>> (2024) [Zugriff: 2026-07-26]

Schwaber, Ken, Sutherland, Jeff (Scrum Guide, 2020): Der Scrum Guide. Der gültige Leitfaden für Scrum: Die Spielregeln, <<https://scrumguides.org>> (2020) [Zugriff: 2026-07-26]

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die angemeldete Prüfungsleistung in allen Teilen eigenständig ohne Hilfe von Dritten anfertigen und keine anderen als die in der Prüfungsleistung angegebenen Quellen und zugelassenen Hilfsmittel verwenden werde. Sämtliche wörtlichen und sinngemäßen Übernahmen inklusive KI-generierter Inhalte werde ich kenntlich machen.

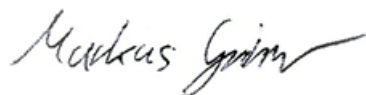
Diese Prüfungsleistung hat zum Zeitpunkt der Abgabe weder in gleicher noch in ähnlicher Form, auch nicht auszugsweise, bereits einer Prüfungsbehörde zur Prüfung vorgelegen; hiervon ausgenommen sind Prüfungsleistungen, für die in der Modulbeschreibung ausdrücklich andere Regelungen festgelegt sind.

Mir ist bekannt, dass die Zuwiderhandlung gegen den Inhalt dieser Erklärung einen Täuschungsversuch darstellt, der das Nichtbestehen der Prüfung zur Folge hat und daneben strafrechtlich gem. § 156 StGB verfolgt werden kann. Darüber hinaus ist mir bekannt, dass ich bei schwerwiegender Täuschung exmatrikuliert und mit einer Geldbuße bis zu 50.000 EUR nach der für mich gültigen Rahmenprüfungsordnung belegt werden kann.

Ich erkläre mich damit einverstanden, dass diese Prüfungsleistung zwecks Plagiatsprüfung auf die Server externer Anbieter hochgeladen werden darf. Die Plagiatsprüfung stellt keine Zurverfügungstellung für die Öffentlichkeit dar.

Gerolsbach, 27.7.2026

(Ort, Datum)



(Eigenhändige Unterschrift)